

PHYS7101: NUMERICAL METHODS AND DATA ANALYSIS · LECTURE 1

Why the computer is not doing algebra

Mathew Smith & Aneta Stefanovska

Foundations for Practical 1

School of Physics
and Astronomy



A surprising failure

```
>> 0.1 + 0.2
```

```
0.30000000000000004
```

- A computer stores numbers to **finite precision** and executes **discrete steps**.
- Every idea in today's lecture — and today's practical — follows from that one fact.
- Floating-point error, algorithmic cost, and step-by-step simulation are all consequences of it, seen from different angles.

Where does numerical error come from?

Computers store numbers in binary, using a finite number of bits.

For example,

$$0.1_{10} = 0.000110011001100110011\dots_2$$

The binary representation is infinite and repeating. A computer therefore has to round it to a finite number of binary digits:

$$0.1_{10} \longrightarrow \underbrace{0.0001100110011\dots_2}_{\text{finite binary representation}}$$

So the number actually stored is only an approximation to the mathematical number (0.1).

The process

decimal input $\longrightarrow$ binary representation $\longrightarrow$ finite storage $\longrightarrow$ decimal display

The displayed decimal number may look exact, but internally the computer is working with its binary approximation.

This is the origin of representation (round-off) error.

Important: It is not the decimal-to-binary conversion itself that is problematic. The problem is that only a finite number of binary digits can be stored.

How does the error enter a calculation?

Consider again our example

$$0.1 + 0.2 = 0.3$$

Mathematically, this is exact. But the computer follows a different sequence:

$0.1_{10} \longrightarrow$ binary approximation $\longrightarrow$ stored value

$0.2_{10} \longrightarrow$ binary approximation $\longrightarrow$ stored value

The arithmetic is then performed on the stored binary values:

$$\text{stored}(0.1) + \text{stored}(0.2)$$

which need not be exactly the stored representation of (0.3).

When converted back for display, we may obtain

$$0.30000000000000004$$

From representation to numerical error

finite representation $\rightarrow$ rounding $\rightarrow$ arithmetic on approximations $\rightarrow$ numerical error

Usually the error is extremely small.

But in a long computation it can:

- accumulate through many operations;
- propagate from one calculation to the next;
- sometimes be amplified dramatically by the mathematical problem itself.

Small numerical errors can therefore become important.

Floating-point representation & error

- Numbers are stored with finite precision.
- **Round-off error**: the small error introduced every time a number is stored or an operation is performed.
- **Truncation error**: the error from approximating an infinite or continuous process with a finite one.
- Kept deliberately distinct – you will meet both again in different guises all module.

In today's practical

You will compute a variance two ways – a naive sum-of-squares formula and Welford's algorithm – and watch the naive version fall apart. This is why.

Computing the variance

For data $(x_1, \dots, x_N)$, the variance can be written as

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N x_i^2 - \underbrace{\left(\frac{1}{N} \sum_{i=1}^N x_i \right)^2}_{\bar{x}^2}$$

This is the sum-of-squares (naïve) formula.

Alternatively,

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Both formulas are mathematically identical.

But they are not necessarily numerically equivalent when evaluated by a computer.

Why?

The first formula subtracts two potentially very large, nearly equal numbers.

This is a recipe for loss of significant digits.

A deceptively simple example

Consider

$$x_i = 10^8 + i, \quad i = 1, \dots, 10.$$

The data are

$$100000001, 100000002, \dots, 100000010.$$

Their variance is small:

$$\sigma^2 = 8.25$$

Yet the quantities appearing in the naïve formula are enormous:

$$\frac{1}{N} \sum x_i^2 \approx 10^{16}$$

and

$$\bar{x}^2 \approx 10^{16}.$$

So we are effectively asking the computer to calculate

$$(10^{16} + \delta_1) - (10^{16} + \delta_2)$$

in order to recover a number of order

$$10^1.$$

The problem

The answer is obtained from the difference between two large approximations.

A tiny relative error in either large quantity can produce a large relative error in their difference.

Catastrophic cancellation

What happens in floating-point arithmetic?

Suppose the computer stores numbers with finite precision.

The quantities

$$\frac{1}{N} \sum x_i^2 \quad \text{and} \quad \bar{x}^2$$

are each calculated with rounding error.

Schematically,

$$A_{\text{computer}} = A + \delta_A,$$

$$B_{\text{computer}} = B + \delta_B.$$

The naïve variance becomes

$$(A + \delta_A) - (B + \delta_B) \quad \text{instead of} \quad A - B.$$

When $A \approx B$, the small difference $(A-B)$ can be dominated by

$$\delta_A - \delta_B.$$

Result

The computed variance may be:

inaccurate

or, in sufficiently extreme cases,

negative

even though a variance mathematically cannot be negative.

This is not a failure of the mathematical formula.

It is a failure of its numerical evaluation.

Welford's algorithm

Instead of computing the difference between two large numbers, update the mean and variance incrementally.

For each new observation (x_k):

$$d_k = x_k - \bar{x}_{k-1},$$

$$\bar{x}_k = \bar{x}_{k-1} + \frac{d_k}{k},$$

$$M_{2,k} = M_{2,k-1} + d_k(x_k - \bar{x}_k).$$

M_2 is the running sum of squared deviations from the current mean.

$$\sigma^2 = \frac{M_{2,N}}{N}$$

$$s^2 = \frac{M_{2,N}}{N-1}$$

population variance sample variance.

Why is this better?

The calculation works with quantities such as

$$x_k - \bar{x}_{k-1},$$

which are of the scale of the variation in the data, rather than forming

$$\sum x_i^2$$

and subtracting $(\bar{x}^2)$.

Avoid the subtraction of nearly equal large numbers.

Three questions to ask of any numerical procedure

Convergence If we refine the calculation, do we get closer to the answer?

Conditioning How sensitive is the problem to small changes in the input?

Stability What happens to numerical errors as the calculation proceeds?

Keep them separate

Conditioning concerns the **problem**; convergence and stability concern the **numerical procedure**.

Convergence: does refinement help?

Suppose the exact answer is y and the numerical answer is y_h . Refine the calculation:

$$h \longrightarrow \frac{h}{2} \longrightarrow \frac{h}{4} \longrightarrow \dots$$

and ask whether

$$y_h \longrightarrow y \quad \text{as} \quad h \longrightarrow 0.$$

Examples:

- smaller time step in an ODE;
- more points in numerical integration;
- more samples in a statistical estimate.

The question

Does making the calculation finer actually bring us closer to the mathematical answer?

Conditioning: how sensitive is the problem?

Change the input slightly:

$$x \longrightarrow x + \delta x.$$

How much does the answer change?

small input change	$\longrightarrow$	$\begin{cases} \text{small output change} & \text{well-conditioned} \\ \text{large output change} & \text{ill-conditioned} \end{cases}$
--------------------	-------------------	---

This is a property of the **mathematical problem itself**.

Think of the variance example

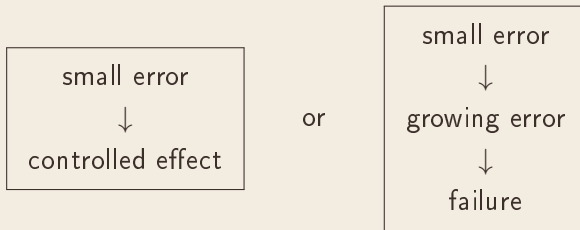
Two large quantities were almost equal. A tiny numerical error in either could have a large effect on their difference.

Stability: what happens to errors?

A numerical calculation inevitably introduces small errors:

rounding $\longrightarrow$ small numerical errors.

The question is what happens next.



Stability

A stable numerical procedure prevents small errors from growing uncontrollably as the calculation proceeds.

Algorithmic complexity, briefly

How does the computational cost grow as the problem gets larger?

$O(n)$: cost grows roughly in proportion to number of data points n ,

$O(n^2)$: cost grows roughly as n^2 .

For example, doubling n gives approximately

$O(n)$: $2 \times$ the work $O(n^2)$: $4 \times$ the work.

Why this matters

The algorithm matters not only because it gives the right answer, but also because of how its computational cost grows.

Describing data

- Populations and samples
- Mean, median and quantiles
- Measures of spread: variance and standard deviation
- Probability distributions and empirical distributions

Why this, why now

These are the basic quantities we will repeatedly compute, estimate and visualise throughout the module.

What is an update rule?

- An **initial-value problem**: a starting point, plus a rule for how it changes.

Numerical integration

- Many problems are described by a differential equation

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$$

- The task is to determine how $y(t)$ evolves from the initial condition.
- Usually, there is no simple analytical expression for $y(t)$.
- Numerical methods approximate the solution at a sequence of discrete points:

$$t_0, t_1, t_2, \dots$$

From a differential equation to an update rule

Suppose

$$\frac{dy}{dt} = f(t, y).$$

At (t_n, y_n) , the differential equation gives the rate of change

$$\frac{dy}{dt} = f(t_n, y_n).$$

A numerical method uses this information to estimate the next value

$$y_{n+1} \approx y(t_n + h).$$

Thus, a numerical integration method can be viewed as an **update rule**:

$$(t_n, y_n) \longrightarrow (t_{n+1}, y_{n+1}).$$

Euler's method

Euler's method uses the slope at the current point:

$$\frac{dy}{dt} = f(t, y).$$

The update is

$$y_{n+1} = y_n + h f(t_n, y_n)$$

with

$$t_{n+1} = t_n + h.$$

In other words:

1. calculate the slope at the current point;
2. multiply it by the step size h ;
3. use it to move to the next point.

Think of it geometrically

Follow the tangent to the solution for one step.

Runge–Kutta methods

Euler uses only one estimate of the slope.

Runge–Kutta methods improve this by evaluating the slope at several points within each step.

For example, the classical fourth-order method, RK4, uses four slope estimates:

$$k_1, \quad k_2, \quad k_3, \quad k_4.$$

The update is

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

where

$$k_1 = f(t_n, y_n),$$

$$k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right),$$

$$k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right),$$

$$k_4 = f(t_n + h, y_n + hk_3).$$

Euler and Runge–Kutta: the basic idea

Slope evaluations		Update
Euler	1	one slope
RK4	4	weighted combination of slopes

Both methods repeatedly apply an update rule:

$$y_n \longrightarrow y_{n+1} \longrightarrow y_{n+2} \longrightarrow \dots$$

Today's practical

Implement Euler and RK4 and watch how the numerical solutions evolve. We will analyse what you observe in the next lecture.

Bridging to the practical

Everything above is what today's practical draws on.

Next lecture – after you have run Euler and RK4 yourselves, we come back to explain what you saw, and ask a more fundamental question: what kind of system are we dealing with?